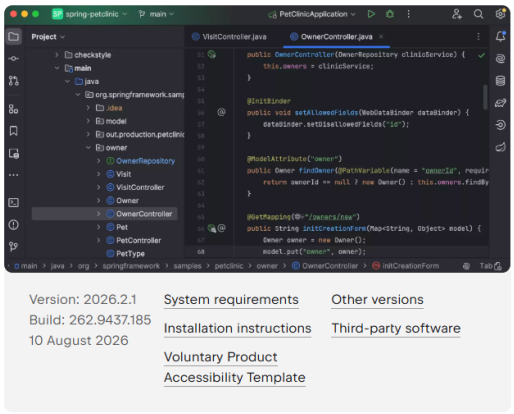
sCOSC2633/2637 Big Data Processing

Week 9 - Lab Task

Scala Maven Project on Intellij IDEA

- For both windows and MacOS users, install IntelliJ IDEA from https://www.jetbrains.com/idea/download/#section=windows





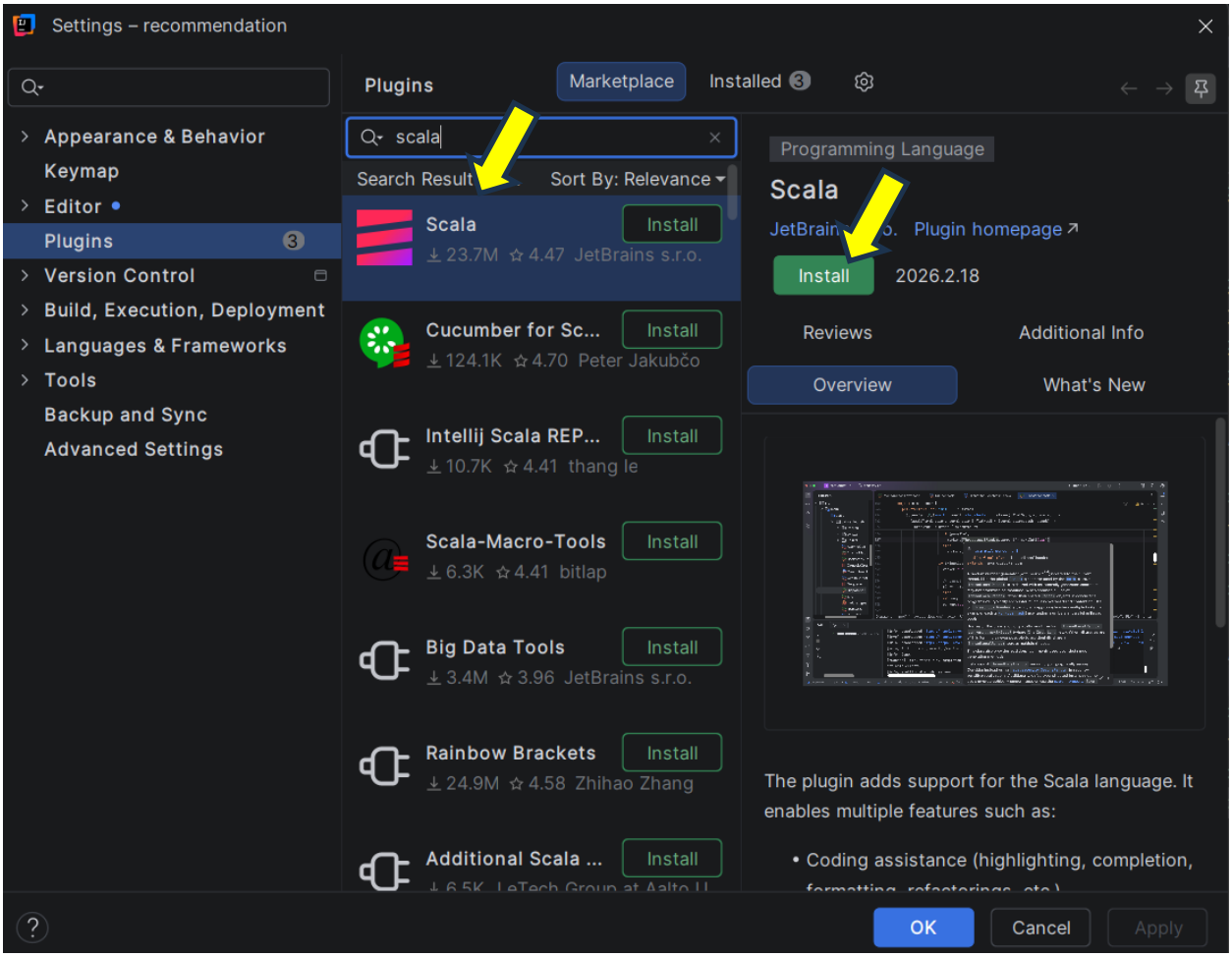
This course only use features from the Intellij IDEA Community Edition. So, it is at no cost.



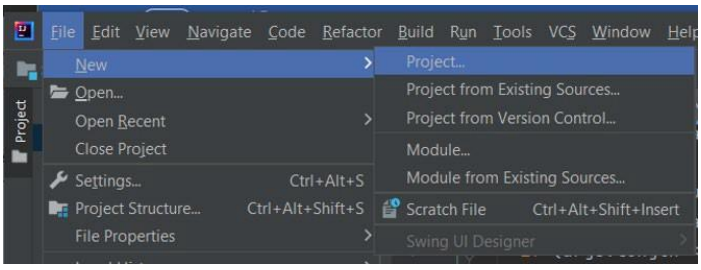
Task: Count characters in a file on HDFS

- Install the required plugins into our IntelliJ.

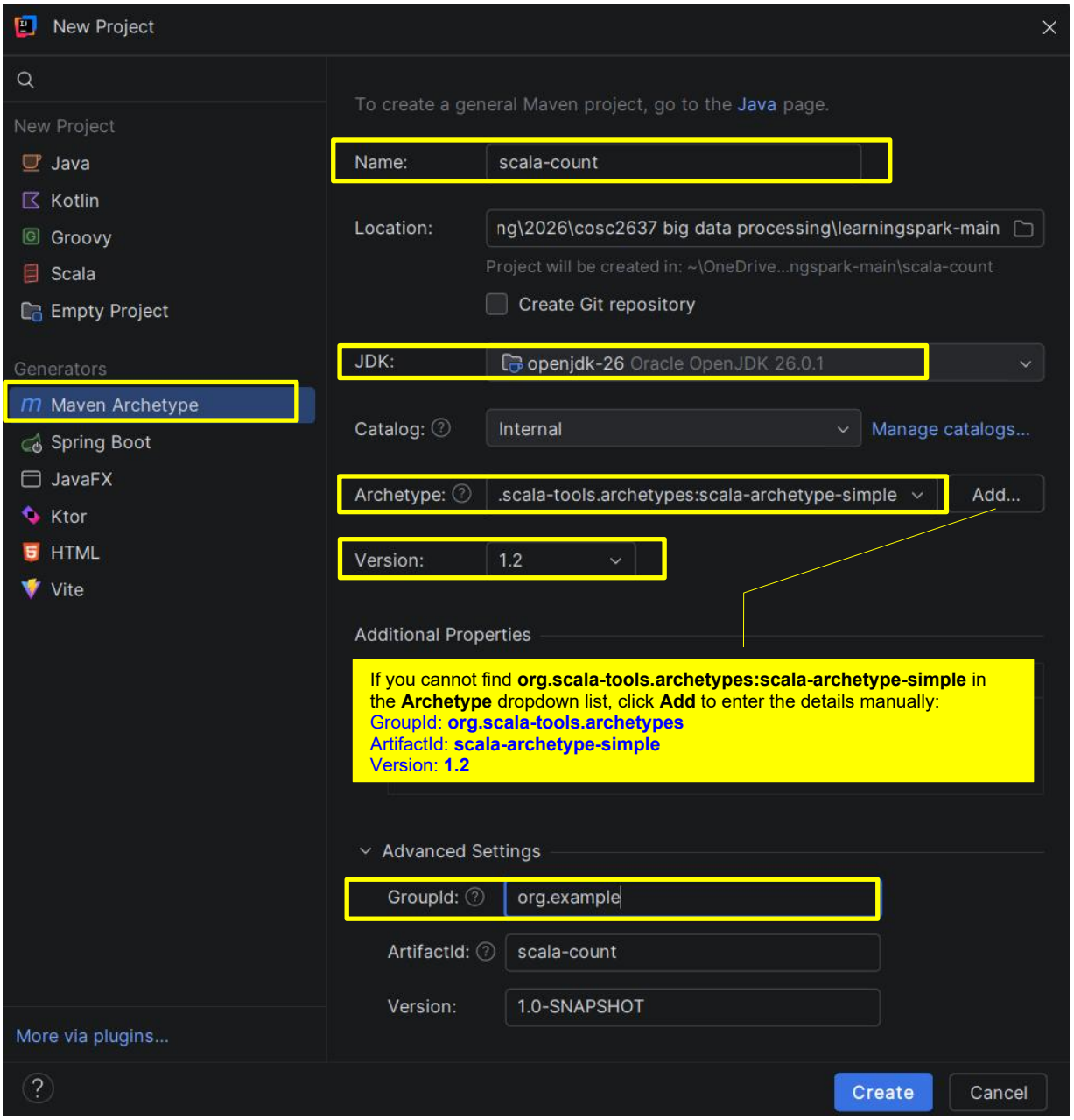
  Select **Plugins** (or go to **File -> Settings -> Plugins**), search for **scala**, and install



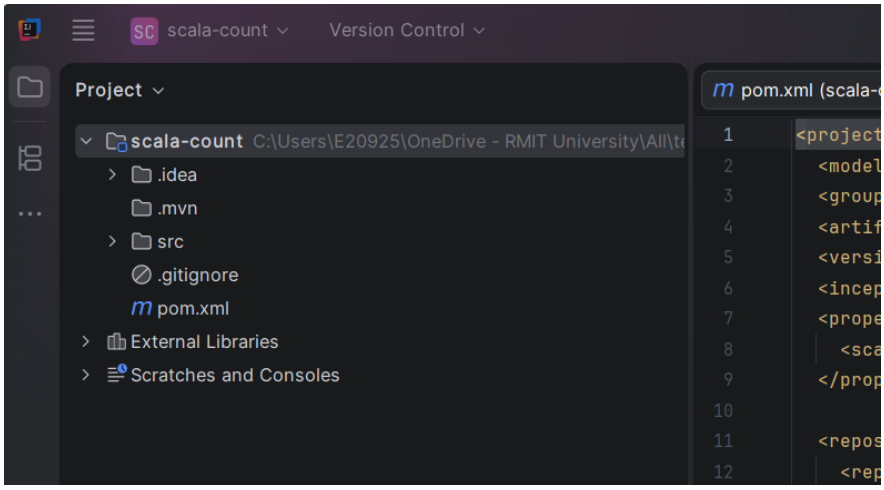
- Now, you can create a **Scala Maven Project**



- In **New Project** window, select **Maven Archetype,** name the project such as **scala-count,** select/text **org.scala-tools.archetypes:scala-archetype-simple** for **Archetype**, name **GroupId** such as **org.example**. Note make sure **JDK** is **JDK1.8**

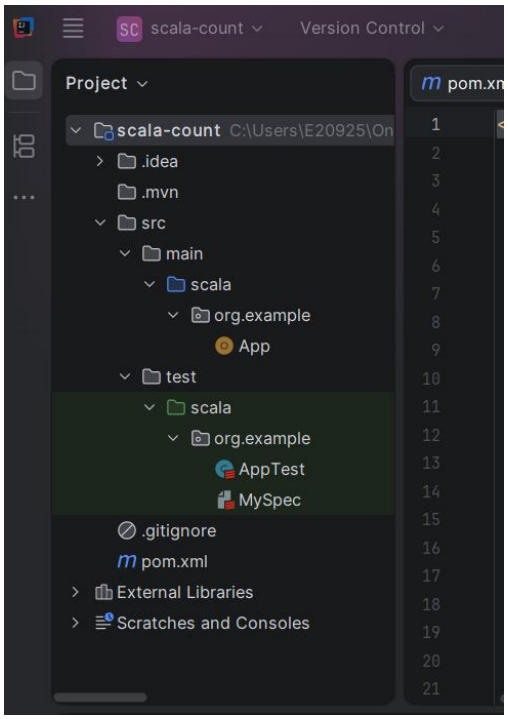


Click **Create**, you will see project **scala-count**

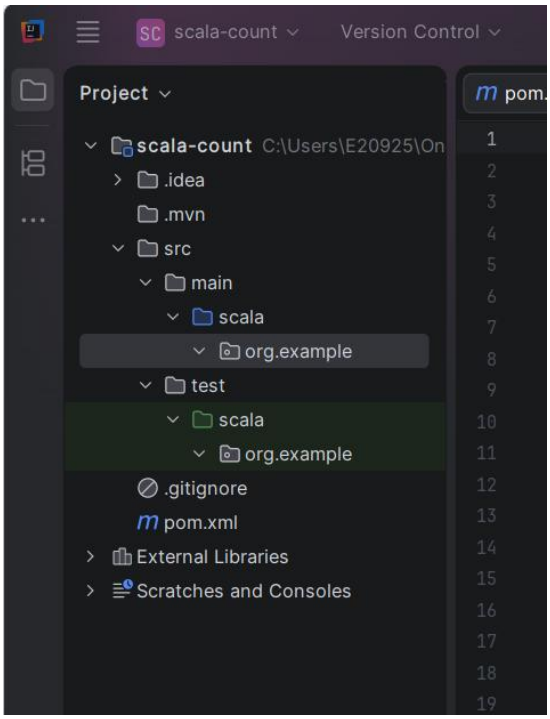


delete

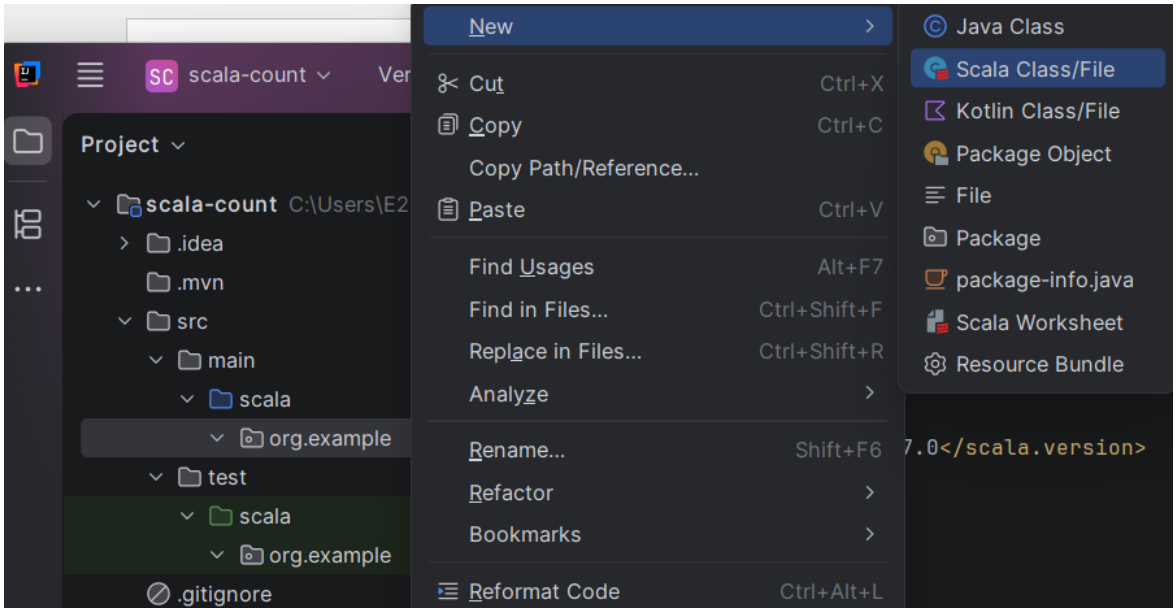
- **main.scala.org.example.App**
- **test.scala.org.example.AppTest**
- **test.scala.org.example.MySpec**



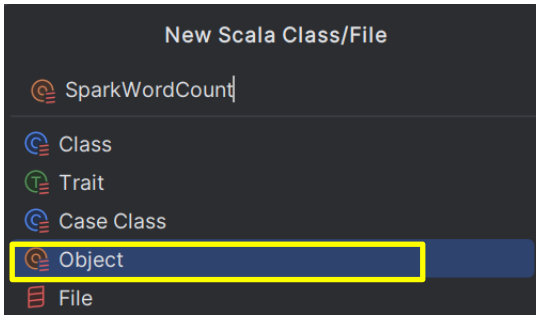




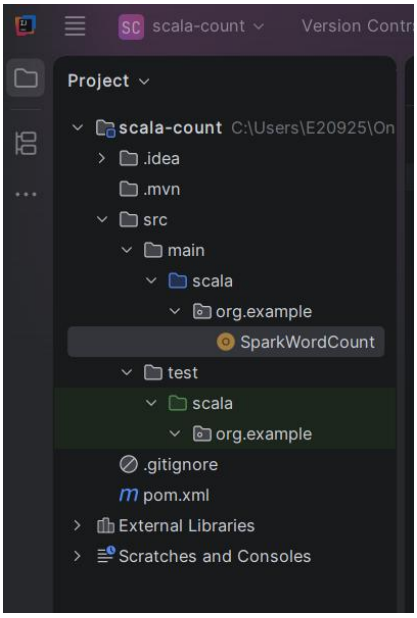
Create a new **scala class** under **main.scala.org.example** package



Name the new Scala Object "**SparkWordCount**", press **enter** key on keyboard.



A new file "**SparkWordCount**" will be created



Copy and paste the code in **SparkWordCount.scala**

pom.xml (scala-count) SparkWordCount.scala

```scala
package org.example

object SparkWordCount {
  def main(args: Array[String]) {
    // create Spark context with Spark configuration
    val sc = new SparkContext(new SparkConf().setAppName("SparkWordCount"))

    // get threshold
    val threshold = args(1).toInt

    // read in text file and split each document into words
    val tokenized = sc.textFile(args(0)).flatMap(_.split(" "))

    // count the occurrence of each word
    val wordCounts = tokenized.map((_, 1)).reduceByKey(_ + _)

    // filter out words with fewer than threshold occurrences
    val filtered = wordCounts.filter(_._2 >= threshold)

    // count characters
    val charCounts = filtered.flatMap(_._1.toCharArray).map((_, 1)).reduceByKey(_ + _)
```

double-click **pom.xml**, change

```
<properties>
  <scala.version>2.7.0</scala.version>
</properties>
```

To

```
<properties>
  <scala.version>2.12.18</scala.version>
</properties>
```

Note this is to make sure the Scala version is consistent with the running environment on AWS EMR as below

```
[hadoop@ip-192-168-25-138 ~]$ spark-shell
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLeve
l(newLevel).
26/09/12 23:04:27 WARN Client: Neither spark.yarn.jars nor spark.yarn.archive is
 set, falling back to uploading libraries under SPARK_HOME.
Spark context Web UI available at http://ip-192-168-25-138.ec2.internal:4040
Spark context available as 'sc' (master = yarn, app id = application_17888488535
68_0022).
Spark session available as 'spark'.
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /___/ .__/\_,_/_/ /_/\_\   version 3.5.6-amzn-2
      /_/

Using Scala version 2.12.18 (OpenJDK 64-Bit Server VM, Java 17.0.20)
Type in expressions to have them evaluated.
Type :help for more information.

scala>
```

Also, add the following dependencies in **pom.xml**

```
<dependency>
  <groupId>org.scala-tools</groupId>
  <artifactId>maven-scala-plugin</artifactId>
  <version>2.11</version>
</dependency>

<dependency>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-eclipse-plugin</artifactId>
  <version>2.5.1</version>
</dependency>

<!-- https://mvnrepository.com/artifact/org.apache.spark/spark-core -->
<dependency>
  <groupId>org.apache.spark</groupId>
  <artifactId>spark-core_2.12</artifactId>
  <version>2.4.5</version>
</dependency>

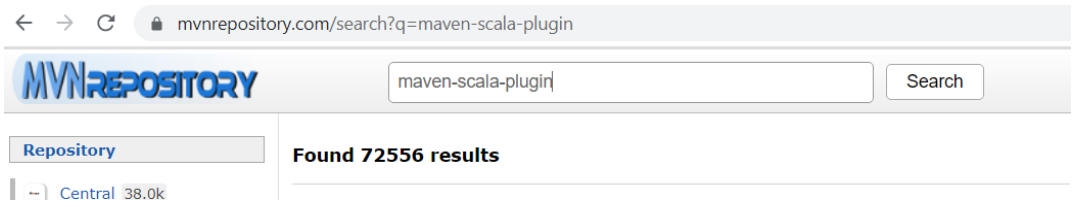
<!-- https://mvnrepository.com/artifact/org.apache.logging.log4j/log4j-core
-->
<dependency>
  <groupId>org.apache.logging.log4j</groupId>
  <artifactId>log4j-core</artifactId>
  <version>2.14.1</version>
</dependency>
```

Open the pom.xml file and press

- Ctrl + Alt + L (Windows/Linux)
- Cmd + Alt + L (macOS)

This will reformat the entire file according to your configured XML code style settings.

The dependency can be extracted from https://mvnrepository.com/.
For “**maven-scala-plugin**”, for example, search for it in the site.



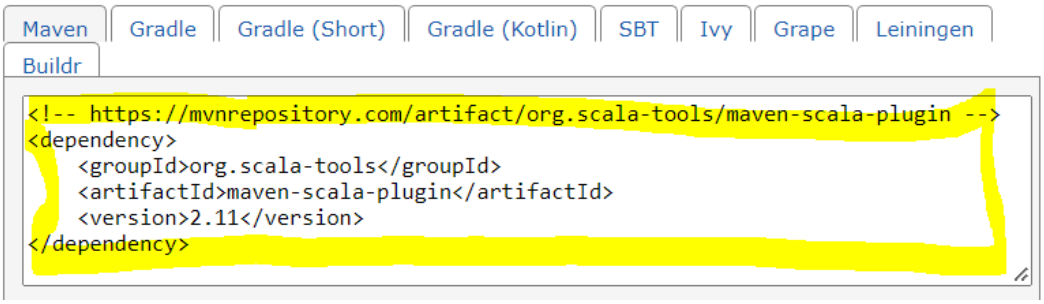
It will return:



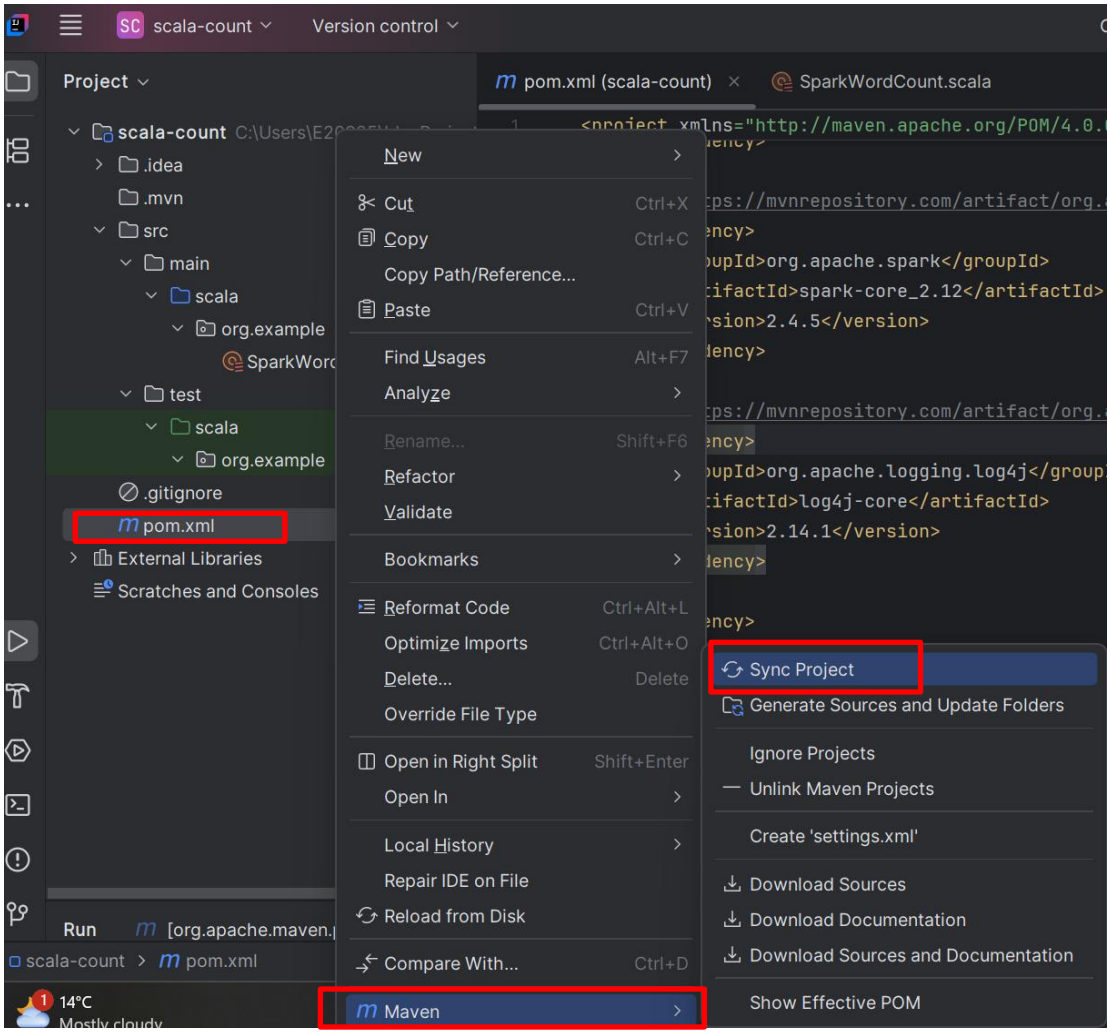
Click “**maven-scala-plugin**”, you will see a list where 2.11 is what we need.



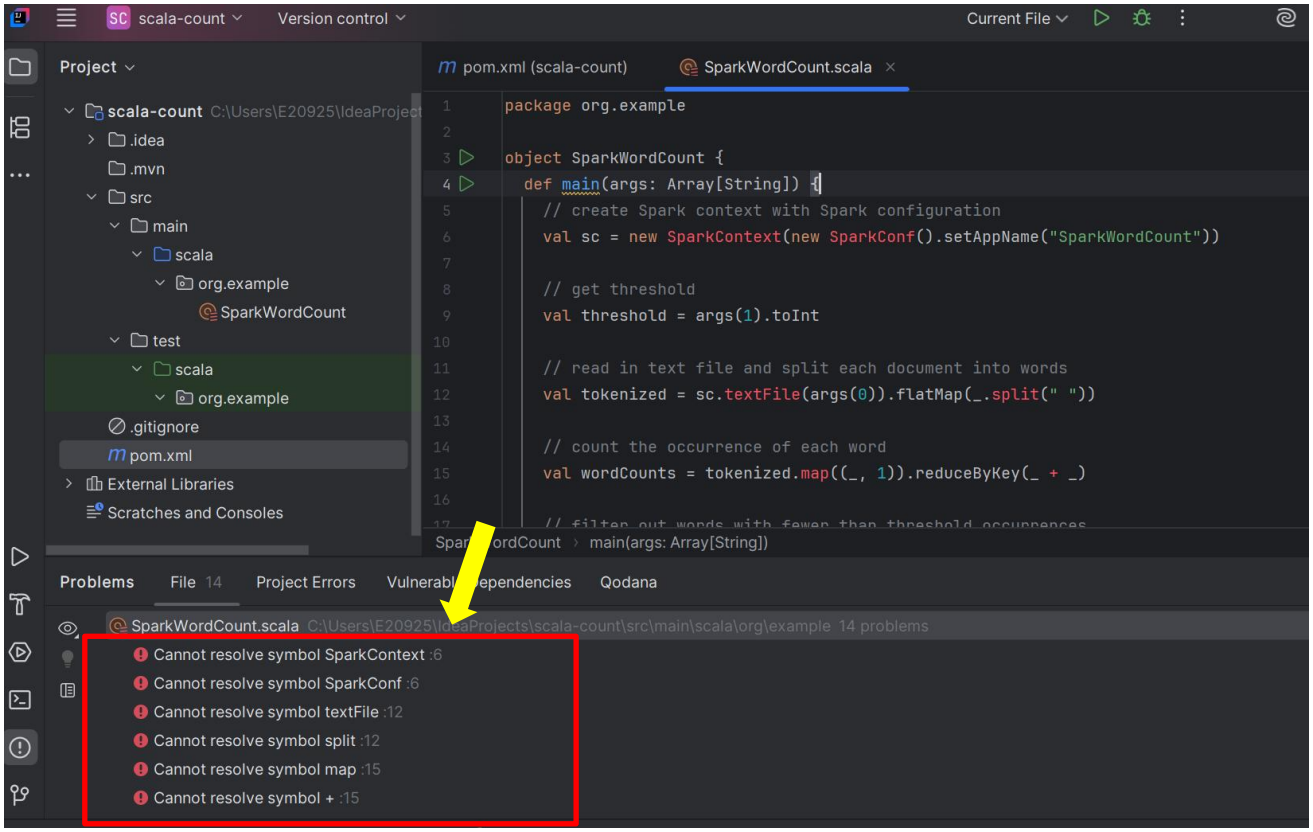
Click 2.11, you will see the following, copy and paste it to **pom.xml**



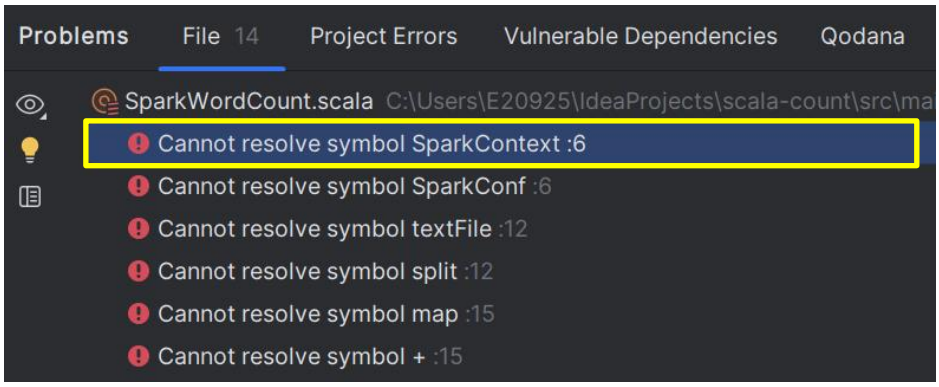
Right click **pom.xml,** select **Maven,** select **Reload Project**



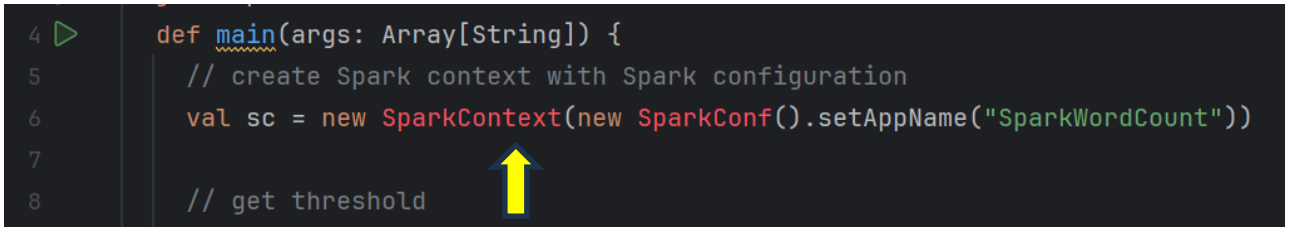
After that, you see error for your code in “**SparkWordCount.scala**”



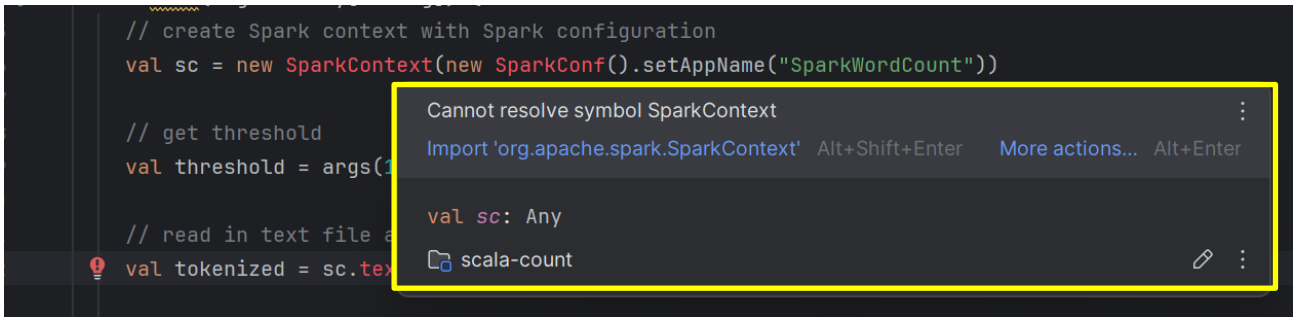
To fix the problem, you need import packages. For error related to **SparkContext**



stop cursor on “**SparkContext**” in **SparkWordCount.scala**,

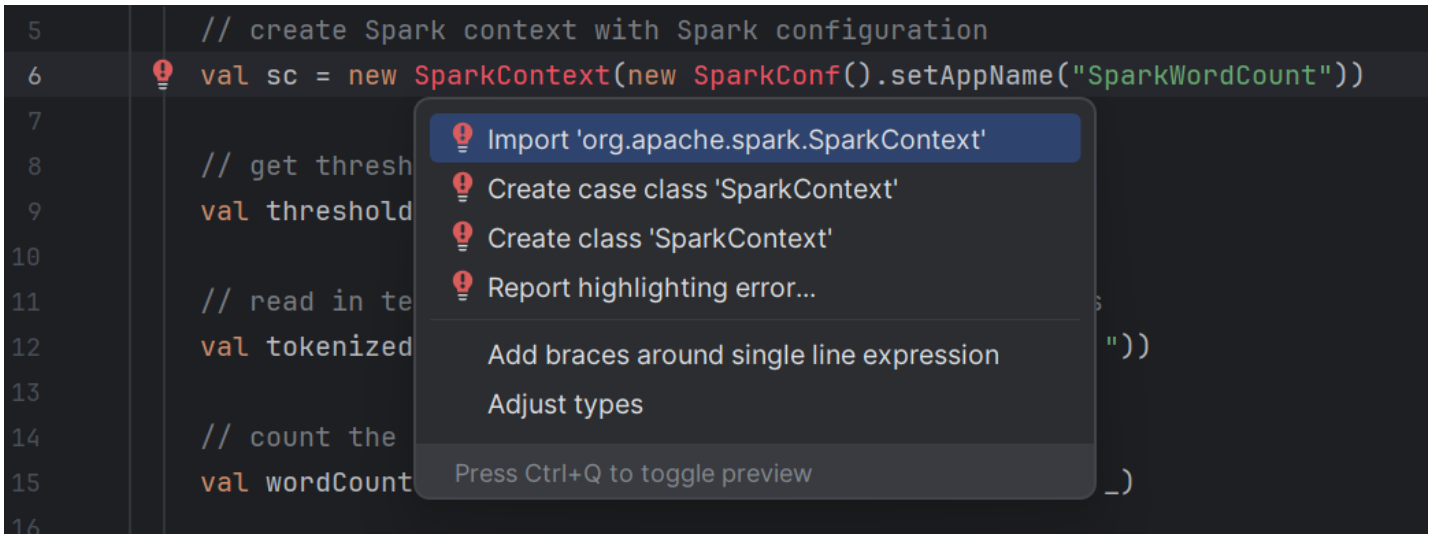


A message dialog will pop up.

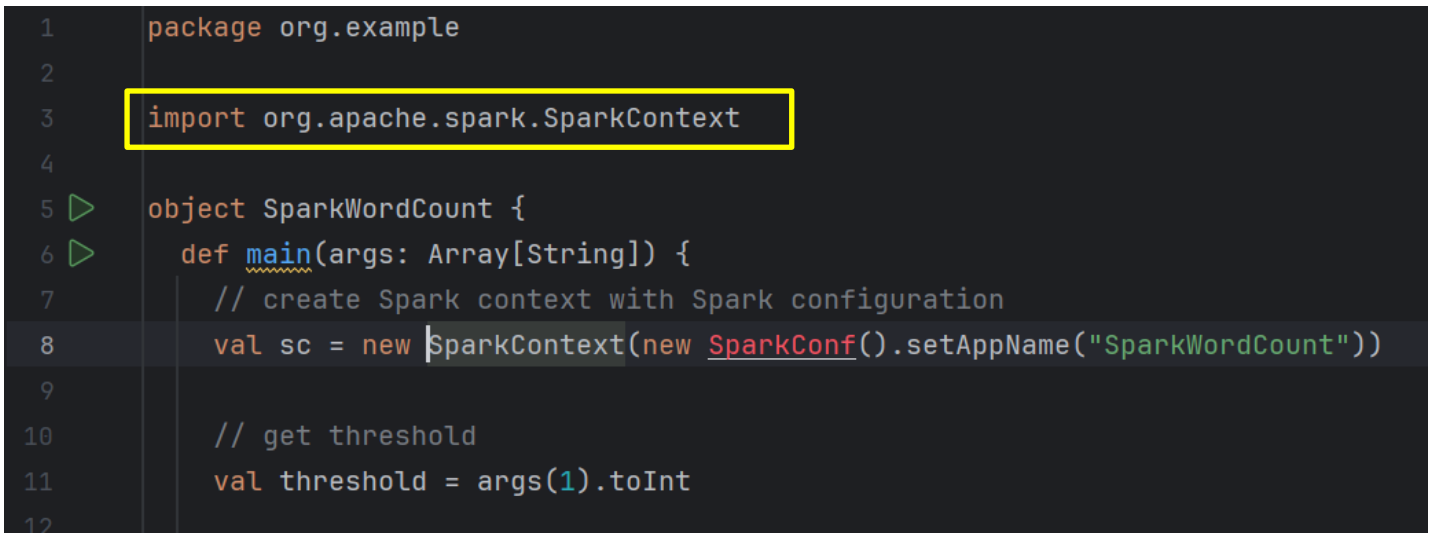


Click "**More actions**", you will see a new message dialog.

Click **"Import 'org.apache.spark.SparkContext'**



Then, a new line of "**import org.apache.spark.SparkContext**" in **SparkWordCount.scala**.

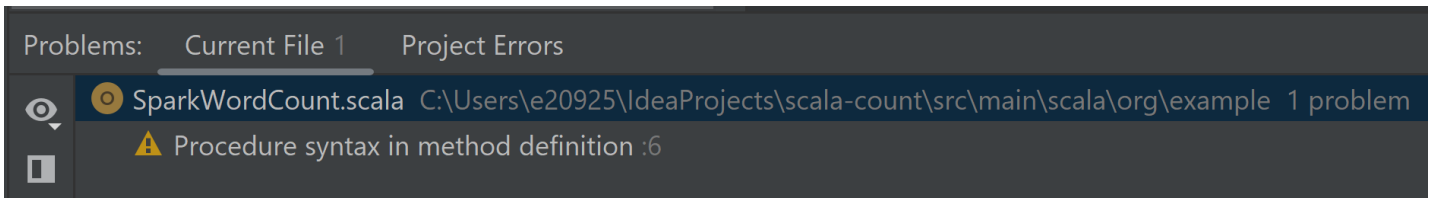


For all red items, process in the similar way. **SparkWordCount.scala** will have a list of

Imports

```
import org.apache.spark.{SparkConf, SparkContext}
```

Then, only one warning is left



This can be solved by change

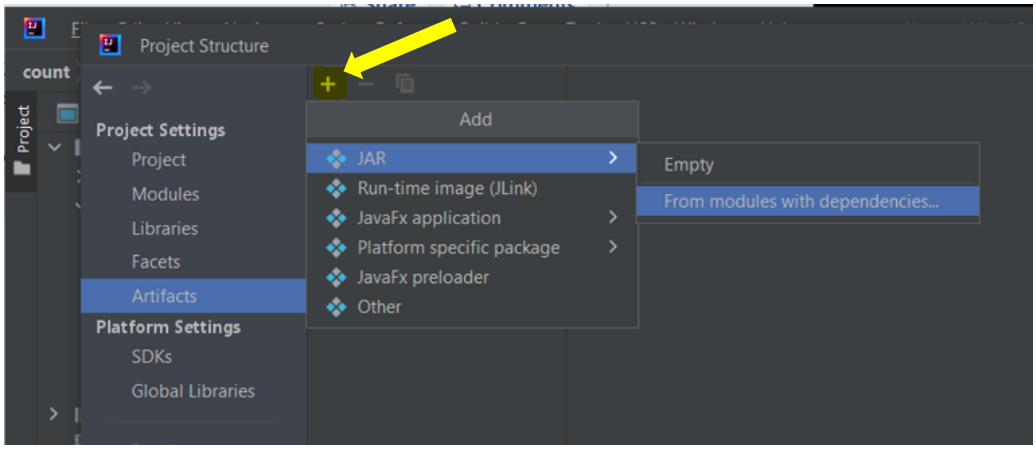
```
def main(args: Array[String]) {
```

to

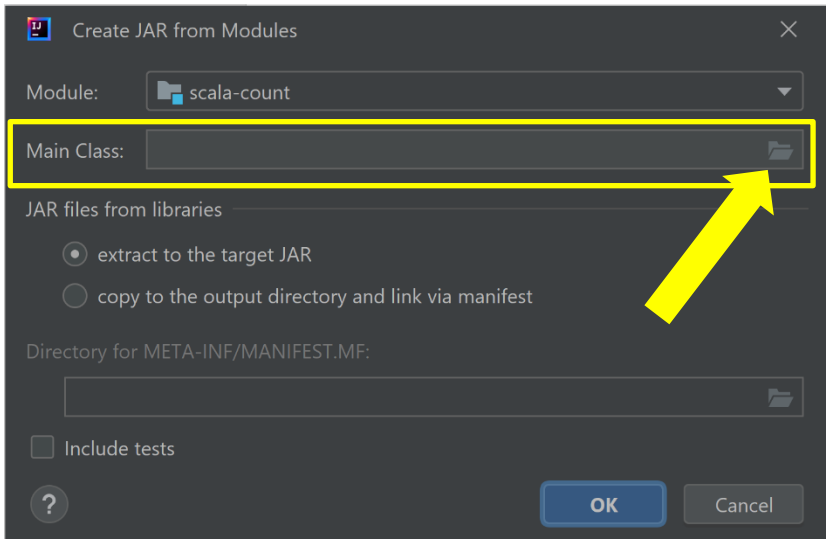
```
def main(args: Array[String]): Unit = {
```

Now, create the .jar file. IntelliJ IDEA enables creation of JAR as an artifact of a project. Do the following steps.

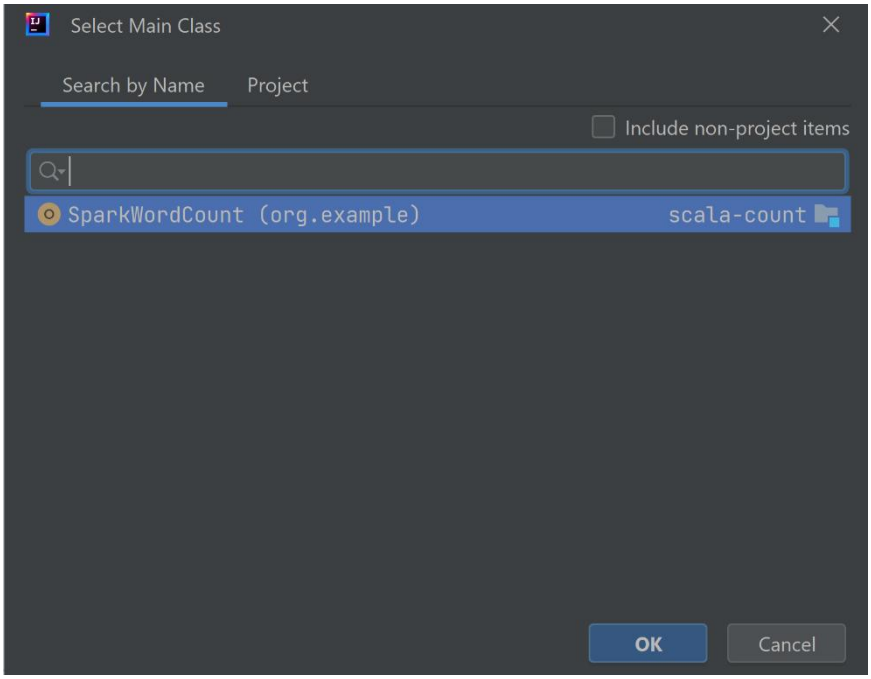
1. From the **File** menu, select **Project Structure...**.
2. From the **Project Structure** window, navigate to **Artifacts** > **the plus symbol + > JAR** > **From modules with dependencies...**.



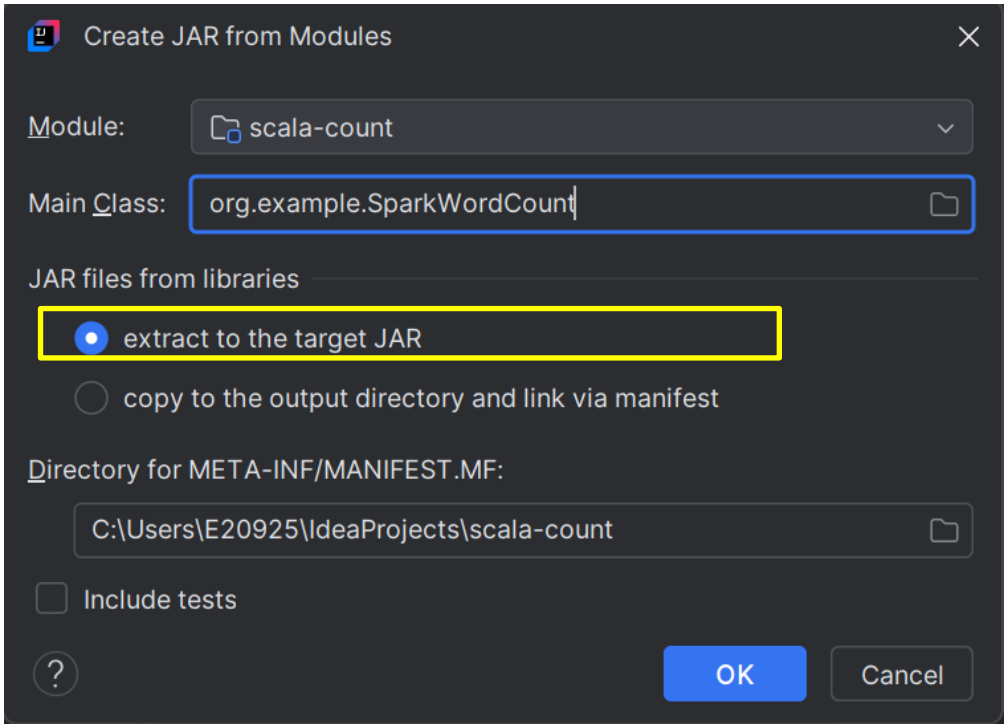
3. In the **Create JAR from Modules** window, select the folder icon in the **Main Class** text box.



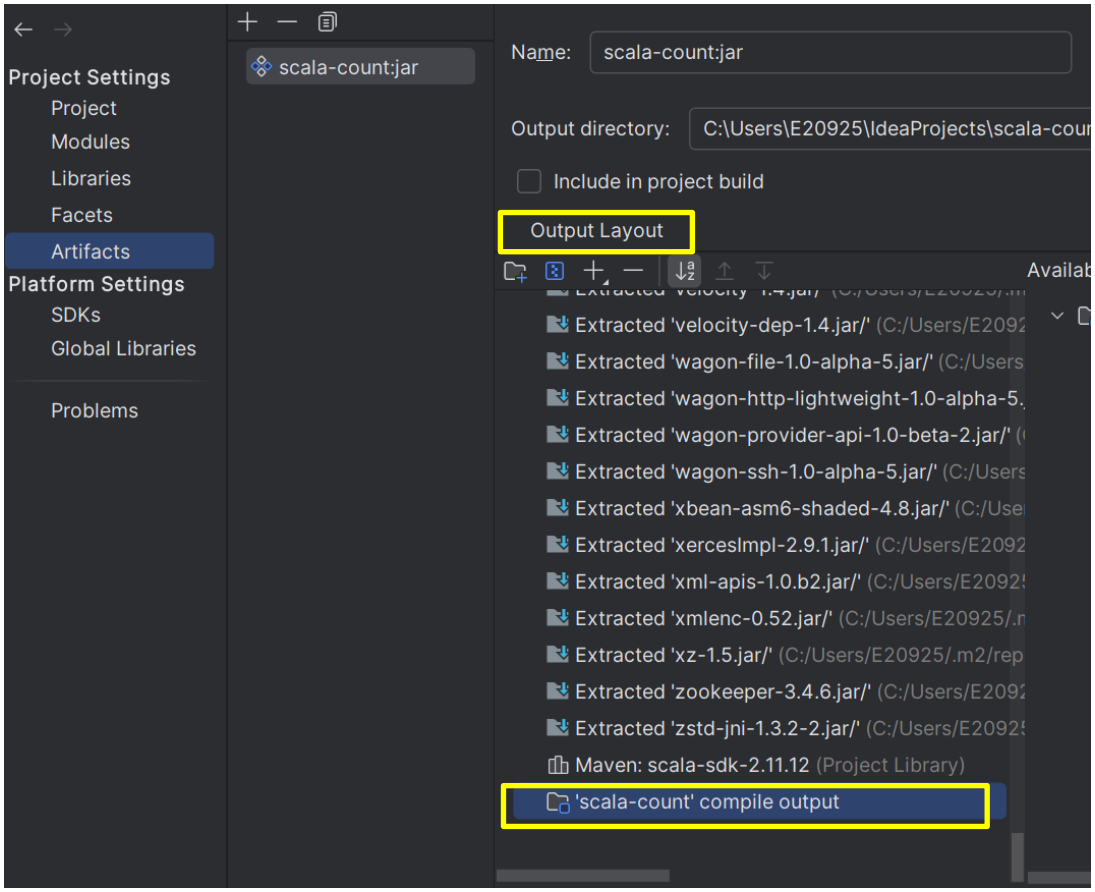
4. In the **Select Main Class** window, select the class that appears by default and then select **OK**.

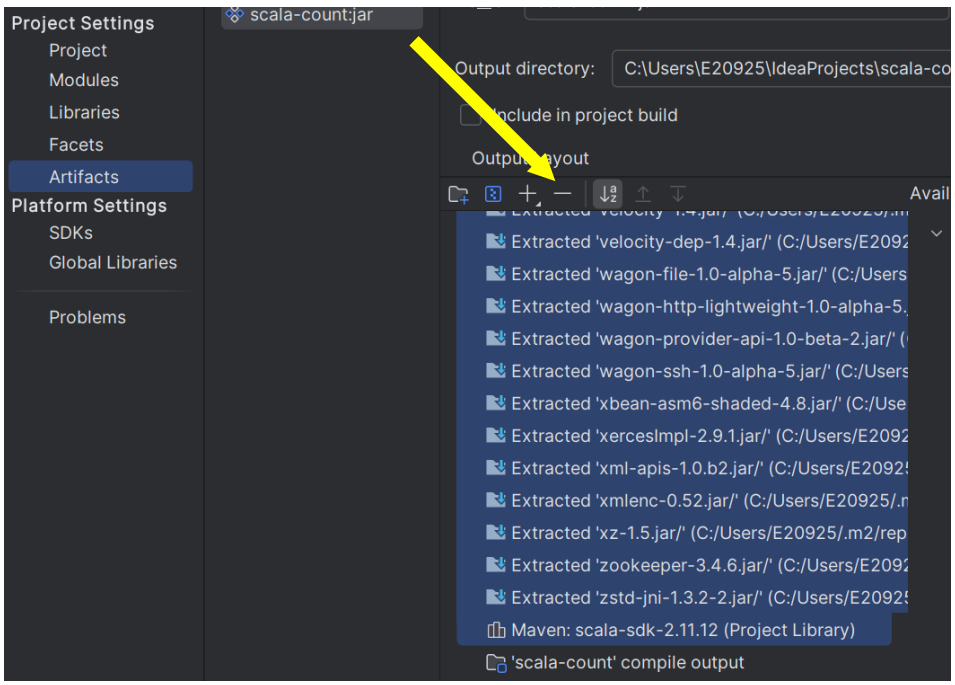


5. In the **Create JAR from** Modules window, ensure the **extract to the target JAR** option is selected, and then select **OK**. This setting creates a single JAR with all dependencies.

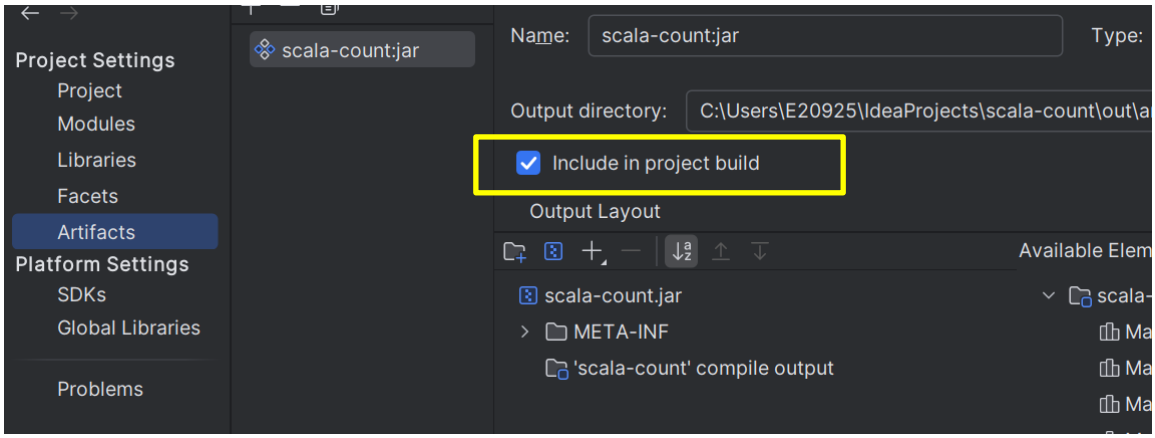


6. The **Output Layout** tab lists all the jars that are included as part of the Maven project. You can select and delete the ones on which the Scala application has no direct dependency. For the application, you're creating here, you can remove all but the last one (**'scala-count' compile output**). Select the jars to delete and then select the negative symbol **-**.



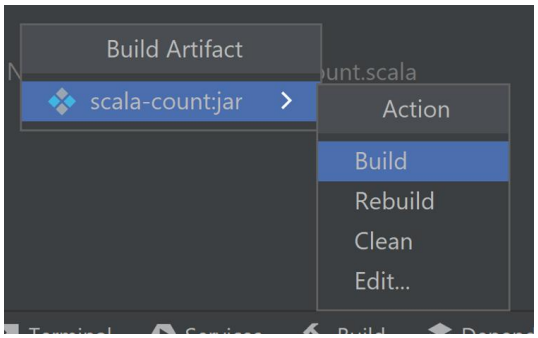


After delete, you will see the following. Ensure sure **Include in project build** checkbox is selected. This option ensures that the jar is created every time the project is built or updated.

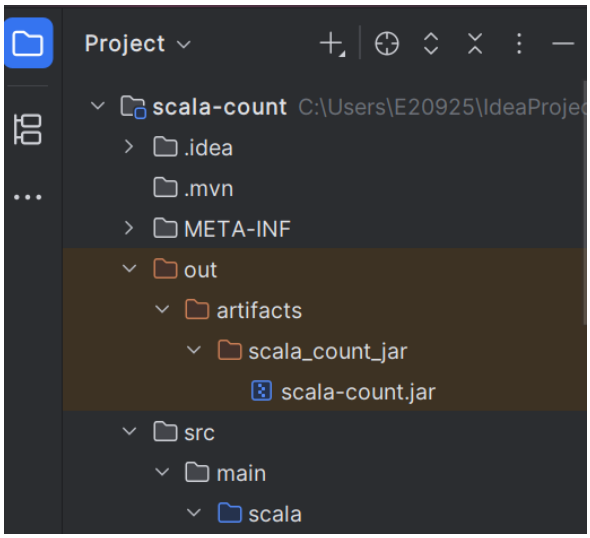


Select **Apply** and then **OK**.

7. To create the jar, navigate to **Build** > **Build Artifacts** > **Build**. The project will be compiled.



The output jar **scala-count.jar** is created under **\out\artifacts\scala-count_jar**



Upload "**scala-count.jar**" to cluster master node as what learned in previous weeks.

Upload a text file such as 3littlepigs.txt to HDFS

```
[hadoop@ip-192-168-16-237 ~]$ ls
3littlepigs.txt  scala-count.jar
[hadoop@ip-192-168-16-237 ~]$ hadoop fs -copyFromLocal 3littlepigs.txt /
[hadoop@ip-192-168-16-237 ~]$ hadoop fs -ls /
Found 5 items
-rw-r--r--   1 hadoop hdfsadmingroup       5485 2023-07-05 11:29 /3littlepigs.txt
drwxr-xr-x   - hdfs   hdfsadmingroup          0 2023-07-05 08:27 /apps
drwxrwxrwt   - hdfs   hdfsadmingroup          0 2023-07-05 08:29 /tmp
drwxr-xr-x   - hdfs   hdfsadmingroup          0 2023-07-05 08:27 /user
drwxr-xr-x   - hdfs   hdfsadmingroup          0 2023-07-05 08:27 /var
[hadoop@ip-192-168-16-237 ~]$
```

In the cluster master node, run the following line:

```
$ spark-submit --class org.example.SparkWordCount --master yarn --deploy-mode client
scala-count.jar hdfs:///3littlepigs.txt 1
```

```
[hadoop@ip-192-168-24-114 ~]$ spark-submit --class org.example.SparkWordCount --
master yarn --deploy-mode client scala-count.jar hdfs:///3littlepigs.txt 1
```

If running successfully, you can see the printed output

```
(T,11), (d,121), (z,3), (",33), (L,5), (p,46), (x,2), (R,1), (B,1), (P,1), (t,15
3), (.,58), (b,46), (h,119), (n,144), (f,45), (j,2), (v,19), (H,5), (F,2), (:,5)
, (,,56), (l,138), (N,6), (r,156), (',19), (s,140), (e,267), (Q,2), (w,66), (G,1
), (a,169), (O,9), (i,146), (y,56), (A,4), (u,79), (I,14), (o,170), (k,35), (!,2
5), (q,3), (-,5), (?,2), (S,3), (C,5), (E,5), (g,68), (W,6), (m,56), (c,65)
```

Two Exercises:

- Can you save the output on HDFS instead of printing it out?
- Can you explain what the output is?

You can post your solution in the discussion forum.

Don't forget to terminate your cluster at end of the lab class!

*Also, if trying out any these exercises outside of the lab class, make sure you terminate your cluster and **DO NOT** leave clusters running idle for days.*

Terminate Cluster